

CONCEPT-ONLY REVISION · NO CODE

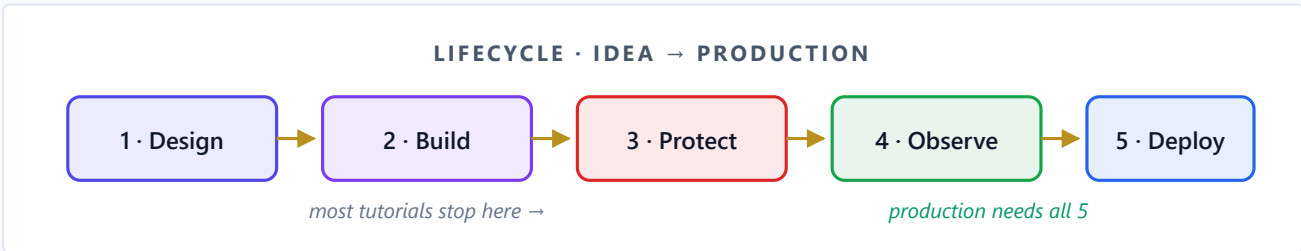
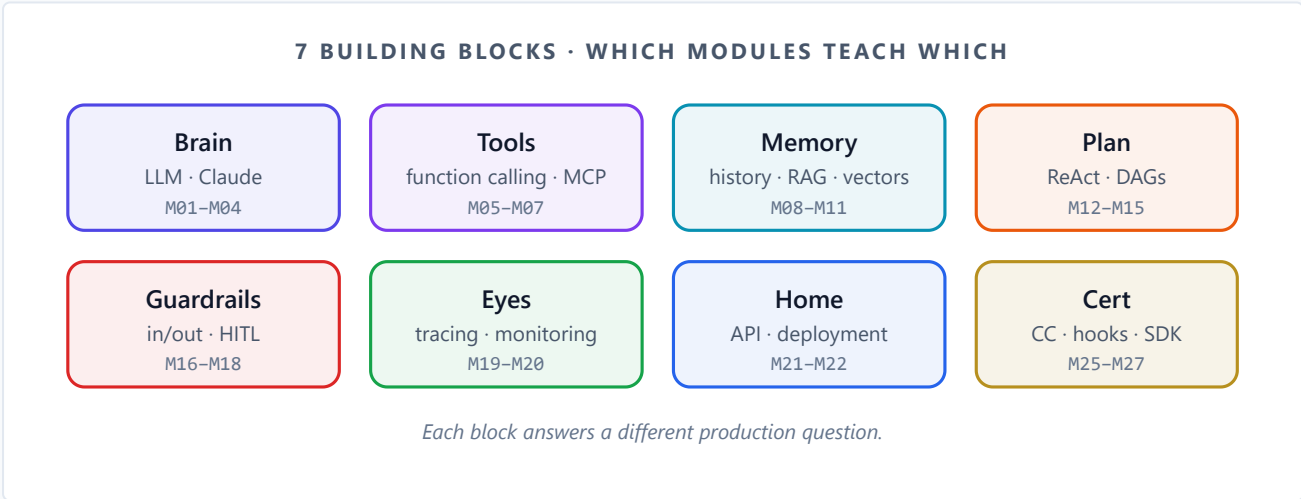
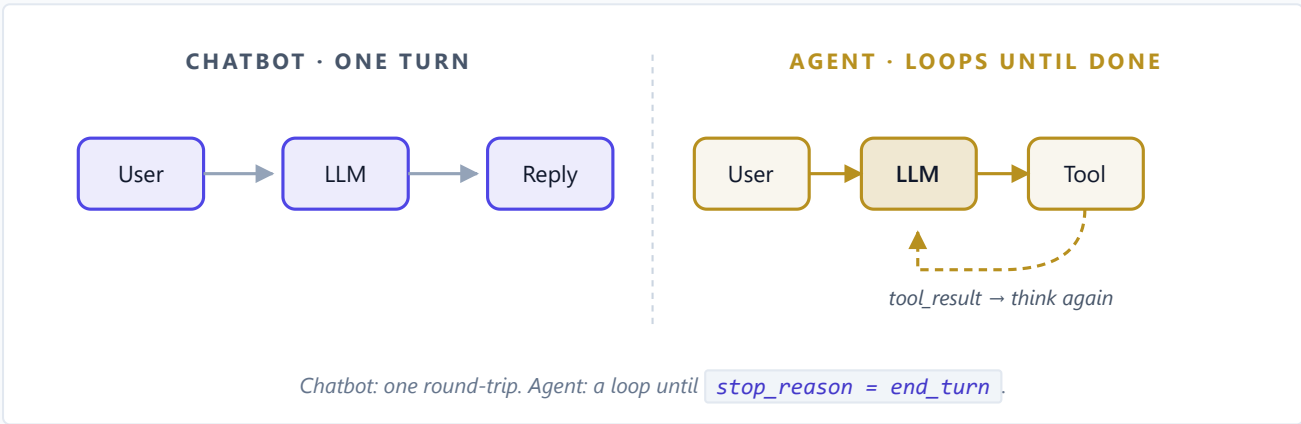
The 30-module course in **one document**

Designed for a few hours of focused revision before the **Claude Certified Architect — Foundations** exam, or before re-reading the full course.

The North Star (M00)

Gateway · code-free

- M00 Agent mental model & lifecycle
- Agent vs chatbot:** a chatbot is one turn (user → LLM → reply). An **agent** is a loop: LLM thinks → calls a tool → reads result → thinks again → repeats until done.
 - The 7 building blocks:** Brain (LLM), Tools (function calling/MCP), Memory (history + RAG + vector DB), Plan (ReAct, decomposition), Guardrails (input/output, HITL), Eyes (tracing, monitoring), Home (API, deployment).
 - The 5 lifecycle stages:** Design → Build → Protect → Observe → Deploy. Most tutorials stop at Build; production needs all five.

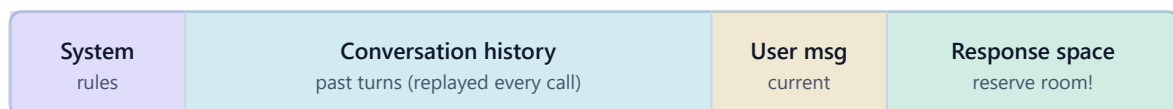


M01 LLM Mental Model

- ▶ LLM = "world's best-read autocomplete" — predicts next token from a probability distribution.
- ▶ Reads input all at once; generates output one token at a time (autoregressive).
- ▶ **Temperature** scales the distribution (0 = deterministic, 1 = creative). **Top-p** keeps the smallest set whose cumulative probability $\geq p$. **Top-k** keeps the K most likely.
- ▶ Mental model: a **thinker**, not a calculator. Probabilistic, not deterministic — needs guardrails.

M02 Tokens

- ▶ Tokens are subword units via BPE (~100K vocab). Common words = 1 token; rare/long words/emoji = several.
- ▶ Three reasons they matter: **cost** (priced per token), **limits** (context window measured in tokens), **performance** (more tokens = slower).
- ▶ **Context window** = system + history + user message + response space. Hitting the limit is hard truncation, not graceful.
- ▶ Output tokens cost more than input tokens. The response itself eats budget — always reserve room.

CONTEXT WINDOW · BUDGET ALLOCATION

⚠ Hard truncation if exceeded — output tokens cost more than input.

M03 Prompts

- ▶ Three roles: **system** (personality, rules), **user**, **assistant**.
- ▶ Patterns: **zero-shot** (just ask), **few-shot** (examples), **chain-of-thought** (reason step-by-step), **role prompting**, **structured output** (XML/JSON), **delimiters** to separate instructions from data.
- ▶ System prompts = "personality programming." Multi-turn = stateless API + you replaying history.

M03B Context Engineering

- ▶ Prompt engineering = writing the message; **context engineering** = **curating everything the model sees**: system + tools + history + retrieved + tool results + current turn.
- ▶ **Four levers**: Add, Compress, Retrieve, Offload — the organizing frame for memory work.
- ▶ **Static vs dynamic**: order matters because prompt caching only hits on identical prefixes.
- ▶ **Position effects**: "lost-in-the-middle" — put critical content at the edges.
- ▶ **Context rot**: stale tool results, superseded instructions, resolved errors poison long sessions.

CONTEXT ENGINEERING · 4 LEVERS

+ Add

Inject more relevant context:
few-shot examples, hints, schemas

≡ Compress

Shrink what's there:
summarize old turns, trim chunks

🔍 Retrieve

Pull on demand, not upfront:
RAG, just-in-time tool calls

➦ Offload

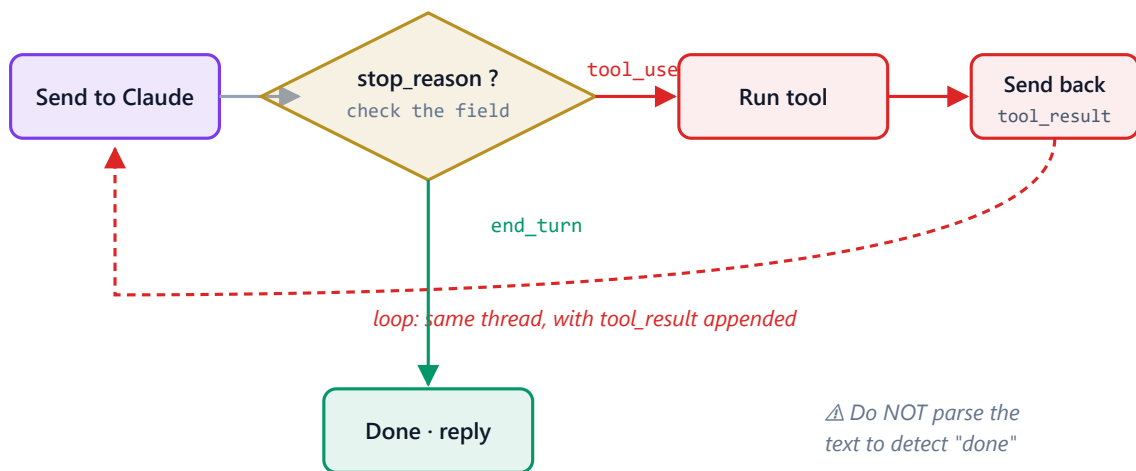
Push state out of the prompt:
scratchpad files, external memory

M04 Structured Output

- ▶ Why agents need it: downstream code needs JSON, not prose.
- ▶ Native tool use is the cleanest path to structured output (Claude returns a typed `tool_use` block).
- ▶ Validate with Pydantic / Zod. On parse failure: feed the error back and retry, don't crash.

M05 Function Calling — the pivotal module

- ▶ A tool definition = `name`, `description`, `input_schema` (JSON Schema). Description quality drives selection accuracy.
- ▶ **Critical:** Claude doesn't run tools — it **asks** to. Your code executes them and returns `tool_result`. Security & control hinge on this.
- ▶ The loop: send → check `stop_reason` → if `tool_use`: run tool, send result back → repeat → exit on `end_turn`.
- ▶ Errors: return them as `tool_result` (not stack traces). Validate args before executing. Never let Claude construct arbitrary commands.

THE TOOL-USE LOOP · stop_reason IS THE CONTROL

⚠ Do NOT parse the text to detect "done"

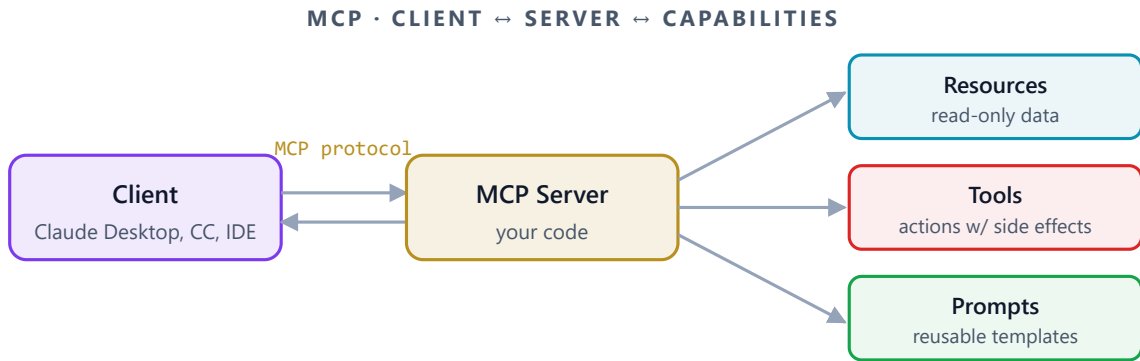
Cert-critical: the loop is controlled by `stop_reason`, never by parsing the model's natural-language output.

M06 Multi-Tool Orchestration

- ▶ **Parallel** tool calls when tools are independent; **sequential** chains when output A feeds B.
- ▶ Selection is description-driven — Claude picks based on what each tool's description claims it does.
- ▶ Tools can be added/removed at runtime (dynamic registration).

M07 MCP — Model Context Protocol

- "USB-C for AI" — a standard protocol so any client (Claude Desktop, Claude Code) can talk to any server.
- Architecture: **Client** ↔ **Server** ↔ **Resources / Tools / Prompts**.
- **Resources** = read-only data (files, DB rows). **Tools** = actions with side effects. **Prompts** = reusable templates.
- Cert anti-pattern: hardcoded API keys in `.mcp.json` → use `${ENV_VAR}` expansion.

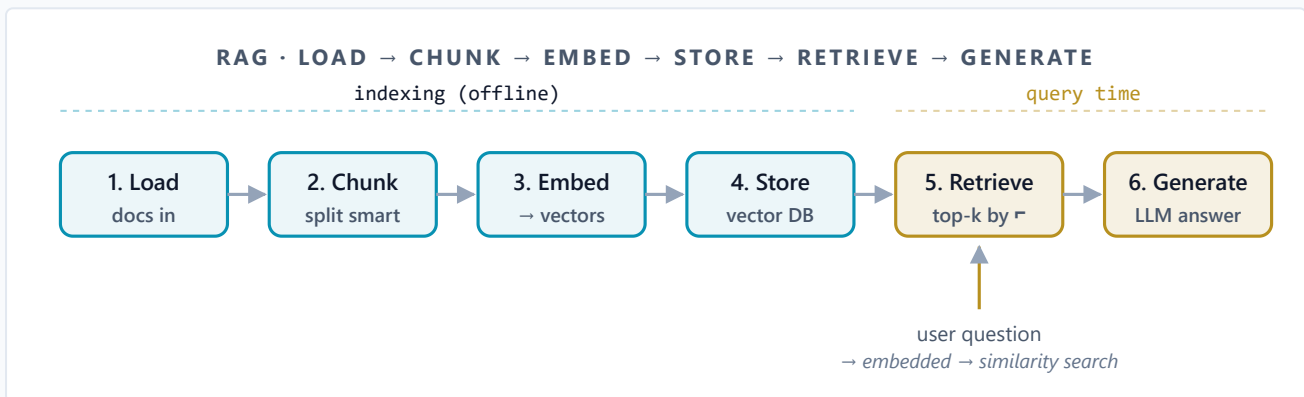


M08 Conversation Management

- ▶ Stateless reality: every API call is fresh. **You** replay history.
- ▶ Strategies: full history (small chats), **sliding window** (recent N), **summarization** (compress old turns).
- ▶ Token budget = system + history + user + reserved response space. Pruning = which turns to keep/drop.

M09 RAG

- ▶ Pipeline: **Load** → **Chunk** → **Embed** → **Store** → **Retrieve** → **Generate**.
- ▶ Embeddings = words/passages mapped to coordinates in meaning-space. **Cosine similarity** = angle between vectors.
- ▶ Chunking strategies: fixed-size (simple), recursive (natural boundaries), semantic (by meaning).
- ▶ Vector DBs: ChromaDB (local), Pinecone (managed), pgvector (Postgres).

**M10 Advanced RAG**

- ▶ **Hybrid search**: BM25 keyword + vector semantic, fused (e.g., reciprocal rank fusion).
- ▶ **Re-ranking**: a second model reorders top-K retrieved chunks by relevance.
- ▶ Query transforms: **HyDE** (generate hypothetical answer, embed that), **multi-query**, **step-back** (more general version first).
- ▶ **Contextual compression**: trim retrieved chunks to only the relevant sentences.
- ▶ Eval metrics: precision, recall, **faithfulness** (answer grounded in sources?).

M11 Multi-Layer Memory

- **Working memory** (scratchpad), **episodic** (vector DB of past interactions), **procedural** (skill library of learned tool sequences).
- Summarization pipeline compresses long sessions; cross-session persistence keeps memory across runs.

MULTI-LAYER MEMORY · 3 TIERS

Tier 1 · Working

scratchpad for current task — fast, small, ephemeral

~ms

Tier 2 · Episodic

vector DB of past interactions — recall by similarity

~10s ms

Tier 3 · Procedural

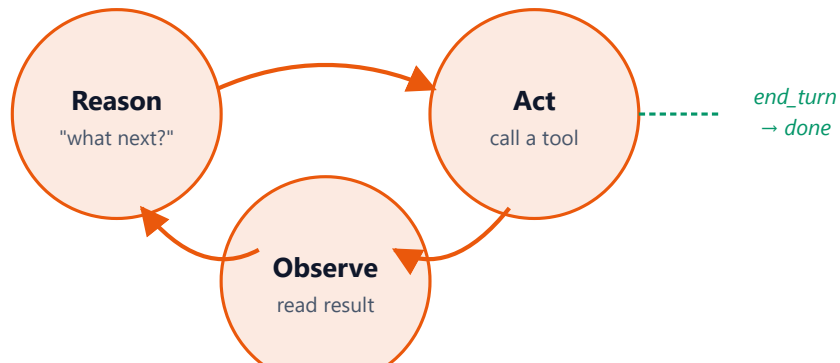
skill library — learned tool sequences, "how to do X"

durable

Each tier serves a different recall pattern.

M12 ReAct Loop

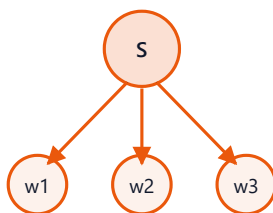
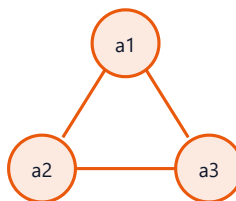
- ▶ **Reason → Act → Observe → Repeat.** The canonical agent shape.
- ▶ Visible thought traces ("think out loud") improve accuracy.
- ▶ Stop conditions: natural termination via `stop_reason: end_turn`; `maxTurns` is a **safety net**, not a primary control.

ReAct LOOP · REASON → ACT → OBSERVE → REPEAT**M13 Planning & Decomposition**

- ▶ Big tasks need a plan. Steps: classify intent → decompose into subtasks → build a **DAG** (dependencies) → execute (parallel where possible).
- ▶ Dynamic tool discovery: pick tools at runtime based on the subtask.

M14 Multi-Agent Systems

- ▶ Patterns: **supervisor/worker** (coordinator delegates), **peer-to-peer** (agents message), **pipeline** (assembly line).
- ▶ Communication: shared state (DB/blackboard) vs message passing (explicit handoffs).
- ▶ Conflict resolution when agents disagree: voting, supervisor decides, escalate to human.

MULTI-AGENT PATTERNS**Supervisor / Worker***delegates & synthesizes***Peer-to-peer***message-passing peers***Pipeline***assembly line, A→B→C*

M15 Code Interpreter / Sandboxed Execution

- Why: LLMs can't reliably do arithmetic / data manipulation — write code, run it, read output.
- Sandboxes: **Docker** (full isolation), **E2B** (managed remote), **Pyodide** (browser).
- Threats: filesystem access, network egress, resource exhaustion. Always isolate.

M15B BUILD: Full agent + subagent system

- Architecture lesson: **single agent with many tools vs coordinator + subagents** — coordinator wins when domains differ (separation of context, fewer tools per agent).
- Coordinator passes context **explicitly**, not inherited. Results flow back and are synthesized.
- Cert relevant: **4–5 tools per agent max**; distribute the rest to subagents.

Track 5 — Guardrails & Safety

M16–M18

M16 Input Guardrails

- **PII detection/redaction** before sending to the model.
- **Prompt injection** types: **direct** (user types malicious instructions), **indirect** (in retrieved docs/web/email), **jailbreaks** (roleplay/encoding tricks). Detect patterns + sandbox untrusted content.
- Schema validation, rate limiting, abuse prevention at the boundary.

M17 Output Guardrails + HITL

- Output checks: hallucination detection (cross-reference sources), toxicity, format validation.
- **Cost controls**: budget caps, token caps, time limits — agent loops can explode.
- HITL gates: **approval** (pause), **modification** (edit), **escalation** (agent recognizes its limits).
- **Circuit breaker**: failure count → threshold → fallback path.
- Cert anti-pattern: sentiment-based escalation. Use **policy gaps, capability limits, thresholds**.

M18 Evaluation & Testing

- Agent eval ≠ unit testing. Metrics: task completion rate, tool selection accuracy, response quality.
- **Claude-as-judge** = automated grading with another LLM call.
- A/B test prompts, tools, strategies. Regression-test with a frozen test set.
- Cert anti-pattern: aggregate-only accuracy → use **per-document-type stratified metrics**.

M19 Tracing & Logging

- ▶ **Trace** = full record of one agent run; **spans** = nested timed sub-operations (LLM, tool, retrieval).
- ▶ Visualize as a waterfall. Log structured fields, never PII.
- ▶ Tools: LangSmith, Langfuse (open source), Arize, OpenTelemetry.

M20 Monitoring

- ▶ Dashboards: latency, token usage, success/failure rates.
- ▶ **Drift detection**: behavior changes over time (prompt regressions, model updates).
- ▶ Alerting: page vs ticket — what's broken vs what's degraded.
- ▶ Feedback loops: production traces → eval set → improvements. Canary / staged rollouts for prompt changes.

Track 7 — Production Deployment

M21 API Design & Deployment

- ▶ Surfaces: REST (simple), WebSocket (bidirectional), **SSE/streaming** (best for slow agent responses — show progress).
- ▶ Containerize with Docker. Cloud targets: AWS Lambda (serverless), Cloud Run (managed containers), Railway.
- ▶ Scale: concurrency, queue-based processing for long jobs.

M22 Cost Optimization

- ▶ Cost anatomy: LLM tokens + tool execution + retrieval + compute.
- ▶ **Prompt caching**: identical prefixes cached → big discount on repeated reads. Order matters (static first, dynamic last).
- ▶ **Model routing**: cheap model for easy, expensive for hard. Cache responses for common queries. Cache embeddings.
- ▶ Token diet: tighten system prompts, constrain output length.

M22B BUILD: Deploy to Local / GCP / AWS

- ▶ Wrap as **FastAPI**: `/query`, `/query/stream` (SSE), `/health`.
- ▶ Why FastAPI: async, auto OpenAPI docs, Pydantic validation. Streaming matters because agents take 5–30s.
- ▶ Docker: multi-stage build, non-root user, secrets via env vars (never baked in).
- ▶ Same agent runs on local Docker, Cloud Run, Lambda — verified with one curl.

M23 Capstone Project Series

- **Domain A:** Healthcare Pre-Authorization (CPT/ICD codes, HIPAA).
- **Domain B:** B2B Ecommerce order tracking (PO lifecycle, carrier APIs, SLAs).
- **Domain C:** Public records / UCC (lien risk, entity resolution, Medallion Architecture).

M24 What's Next — Agent Frontier

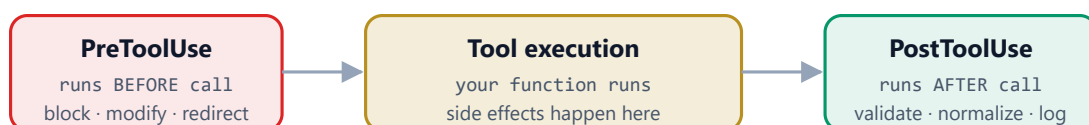
- Agent-to-agent protocols, agent marketplaces, computer use, extended thinking.
- Building responsibly: alignment, oversight, ethics.

M25 Claude Code Mastery (Cert Domain 3, ~20%)

- ▶ **CLAUDE.md hierarchy** (cascades like CSS): user-level (`~/ .claude/`) → project-level (`.claude/`) → directory-level. More specific wins. `@import` shares rules.
- ▶ **Slash commands vs skills**: commands are speed-dial buttons (manual); skills are auto-invoked by description match. Skills support `context: fork` (isolated context) and `allowed-tools` restrictions.
- ▶ **Plan mode**: Claude drafts → you approve → executes. Use for unfamiliar codebase, risky/complex changes. Direct execution for small, well-understood tasks.
- ▶ Patterns: **TDD** (red-green-refactor with Claude), **interview pattern** (tell Claude to ask **you** questions first), **concrete examples** (give 2–4 then generalize).
- ▶ Built-ins: Glob (find) → Read (understand) → Edit (change) → Bash (test). Grep for patterns.

M26 Hooks, Sessions & Agent SDK (Cert Domain 1, ~25%)

- ▶ **Agent SDK** `query()` : handles the loop, streaming, tool execution automatically. Messages API = manual loop; SDK = managed loop.
- ▶ `stop_reason` is the loop control: `tool_use` → continue, `end_turn` → stop, `max_tokens` → handle gracefully. **NEVER parse natural language** to detect "done."
- ▶ **Hooks** = deterministic enforcement (CODE), unlike prompts (probabilistic). **PreToolUse**: before tool call (block/modify/redirect). **PostToolUse**: after (validate/normalize/log).
- ▶ Use hooks for critical business rules (refund caps, PII, compliance). Use prompts for style/tone.
- ▶ **Sessions**: named (`--session`), `--resume` , `fork_session` for parallel exploration without polluting main context.

HOOKS · DETERMINISTIC ENFORCEMENT AROUND TOOL CALLS

Hooks = code (100% reliable)

Critical rules belong here, NOT in prompts (which are probabilistic).

Cert anti-pattern: prompt-based enforcement of critical rules. Use hooks instead.

M27 Cert Exam Prep — Domain 4 essentials

- ▶ `tool_choice` : `'auto'` (Claude picks if needed), `'any'` (must use **some** tool), forced specific tool. Know when each fits.
- ▶ Schema design: required vs optional, enums with `'other'` + freetext detail, nested objects.
- ▶ **Validation-retry**: on failure, append the specific error → re-request → up to ~3 tries. Track `detected_pattern` — if same field fails 3 times, escalate to human.
- ▶ Multi-pass review: pass 1 per-file (style/bugs); pass 2 cross-file integration. Use **separate sessions** per pass.

M27B Domain 5.6 Deep Dive (under-covered cert topic)

- ▶ **Provenance:** structured `{claim, source, confidence}` objects (not prose with parens). Retraction must propagate.
- ▶ **Temporal data:** `{value, valid_from, valid_to, source}`. Distinguish "as-of" vs current queries. Watch the missing-`valid_to` bug.
- ▶ **Stratified sampling for human review:** take N from each confidence bucket — beats top-N and uniform random.
- ▶ **Field-level confidence** beats document-level for high-stakes extraction; composes with stratified sampling.
- ▶ **Synthesis output buckets:** `established / contested / single-source / temporal-warning`. Paraphrase = agreement.

The 18 Anti-Patterns — Memorize These

These appear across all 5 cert domains. Each card shows the **anti-pattern** on top and the **correct approach** below.

Domain 1 — Agentic Architecture

#1 **DON'T** Parse natural language to end the loop

#1 **DO** Check `stop_reason` — the source of truth for control flow

#2 **DON'T** Use arbitrary iteration caps as the primary stopping mechanism

#2 **DO** Natural termination via `stop_reason` ; `maxTurns` is only a safety net

#3 **DON'T** Prompt-based enforcement for critical business rules

#3 **DO** Programmatic hooks (PreToolUse / PostToolUse) — deterministic, not probabilistic

#4 **DON'T** Sentiment-based escalation ("user sounds upset")

#4 **DO** Escalate on policy gaps, capability limits, and explicit thresholds

#5 **DON'T** Self-reported confidence scores from the model

#5 **DO** Structured criteria + programmatic checks

Domain 2 — Tool Design / MCP

#6 **DON'T** Return generic errors like "Operation failed"

#6 **DO** Structured: `isError` , `errorCategory` , `isRetryable` , context

#7 **DON'T** Return empty results for access failures

#7 **DO** Distinguish "denied" from "genuinely empty"

#8 **DON'T** Pack 18+ tools into one agent

#8 **DO** 4–5 tools per agent; distribute the rest to subagents

#9 **DON'T** Hardcode API keys in `.mcp.json`

#9 **DO** Use `${ENV_VAR}` expansion

Domain 3 — Claude Code Config

#10 **DON'T** Put personal prefs in project-level CLAUDE.md

#10 **DO** User-level for personal; project-level for team standards

#11 **DON'T** Use commands for complex tasks needing context isolation

#11 **DO** Use skills with `context: fork` + `allowed-tools`

#12 **DON'T** Same-session self-review in CI

#12 **DO** Separate sessions for generate vs review

Domain 4 — Prompt Engineering / Validation

#13 **DON'T** Vague instructions like "be thorough"

#13 **DO** Explicit, measurable criteria (counts, formats, constraints)

#15 **DON'T** Generic retry ("try again")

#15 **DO** Append the specific error (which field, expected vs actual)

#14 **DON'T** Assume `tool_use` guarantees semantic correctness

#14 **DO** Schema validation + business-rule validation

Domain 5 — Context & Reliability

#16 **DON'T** Progressive summarization of critical details

#16 **DO** Immutable case-facts blocks for must-keep details

#18 **DON'T** Skip provenance tracking in multi-agent systems

#18 **DO** Track source, confidence, timestamp, agent ID for every claim

#17 **DON'T** Aggregate-only accuracy metrics

#17 **DO** Per-document-type stratified metrics

The 7 mental models to carry into the exam

- 1 **The agent is just a loop** around a stateless LLM. Everything else is plumbing for that loop.
- 2 **Context is curated, not just written.** Add / Compress / Retrieve / Offload.
- 3 **Probabilistic $\neq$ enforceable.** Critical rules go in code (hooks, validators), not prompts.
- 4 `stop_reason` **is the source of truth** for control flow. Never parse text to decide what to do next.
- 5 **Tools are *your* functions**, called on Claude's request. Validate args, structure errors, never execute arbitrary code.
- 6 **Production = Build + Protect + Observe + Deploy**, not just Build.
- 7 **Provenance matters:** every claim from a multi-agent system needs source + confidence + timestamp.